README – Onward!

Title: Onward!
Author: Evan Demitriou
Engine: PuzzleScript

Game Overview

Onward! is a tile-based puzzle game built in PuzzleScript. The player must navigate from a starting position to an exit while managing irreversible movement, unlocking doors with keys, avoiding hazards, and manipulating blocks. The game emphasizes forward planning, as you can't go back, therefore you must go onward! Because movement is limited by a trail mechanic that permanently blocks previously visited tiles, players must think carefully before committing to a path. The game contains four levels, each introducing and combining mechanics to increase complexity.

Core Mechanic

- Trail / Irreversible Movement System: Every time the player moves onto a background tile, that tile becomes a “Trail”. At the end of the turn, all Trail tiles convert into Spent tiles. Spent tiles cannot be stepped on again. Attempting to move onto a Spent tile cancels the move (acts like you didn't move at all).
- This creates a one-directional, no-backtracking experience. Every step is permanent, forcing deliberate route planning rather than trial-and-error wandering.

Implementation (code summary):

```
[ Player Background ] -> [ Player Trail ]
late [ Player Trail ] -> [ Player ]
late [ Trail ] -> [ Spent ]

[ > Player | Spent ] -> [ > Player | Spent ] CANCEL
```

Ancillary Mechanics

1. Keys and Doors (State Change / Gating)

The game includes three key-door pairs:

- KeyA opens DoorA
- KeyB opens DoorB
- KeyC opens DoorC

Keys can be pushed by the player. When a key is pushed into its corresponding door, both the key and the door disappear, clearing the path.

Example rule:

```
[ > Player | KeyA | DoorA ] -> [ > Player | Background | Background ]
```

2. Hazards (Failure Condition)

Hazards act as lethal tiles. If the player steps onto a hazard, the level restarts. If a block is pushed onto a hazard, the block is destroyed. Keys cannot be destroyed as they are used to open doors (you wouldn't want to break the game now, would you?).

Implementation:

```
late [ Player Hazard ] -> RESTART
late [ Block Hazard ] -> [ Hazard ]
```

3. Pushable Blocks (Spatial Manipulation)

Blocks can be pushed by the player in the direction of movement. Blocks cannot be pulled and only move when pushed. This is used to push past areas, allowing routes to be made.

Example rule:

```
[ > Player | Block ] -> [ > Player | > Block ]
```

Collision Layers

Collision layers are structured as follows:

```
Background
Exit, Trail, Hazard, Spent
Player, Wall, DoorA, DoorB, DoorC, KeyA, KeyB, KeyC, Block
```

This ensures:
Player cannot overlap walls, doors, keys, or blocks, can overlap hazards and spent tiles (with rules determining consequences), and Blocks behave as solid objects.

Win Condition

The player wins when reaching the exit:

```
some Player on Exit
```

Each level contains at least one valid solution.

Level Design Breakdown

Level 1: Trail

Introduces the core mechanic (the trail).
Players learn that movement is permanent and cannot be reversed.

Level 2: Locks and Keys

Introduces key-door mechanics. Players must sequence key usage carefully without trapping themselves.

Level 3: Hazards

Focuses on navigating the extremely hazardous environment while preserving safe movement paths. One wrong move and back to the start of the level.

Final Level: Blocks & Friends

Combines (like a final exam):

- Trail mechanic
- Multiple keys and doors
- Hazards
- Pushable blocks

Controls

Arrow Keys or WASD: Move player (up, down, left, and right)

R: Restart level

Z: Undo (PuzzleScript default, if enabled)

Have Fun!